



Pipeline Parallelism in Transformers

Splitting Layers Across GPUs for Efficient Training

A technical exploration of how to efficiently scale transformer models beyond single-GPU memory constraints by distributing computation across multiple devices.

Why Pipeline Parallelism?

Memory Limitations

Transformer models exceed **single-GPU memory** capacity, especially as we scale to billions of parameters

Data Parallelism Drawbacks

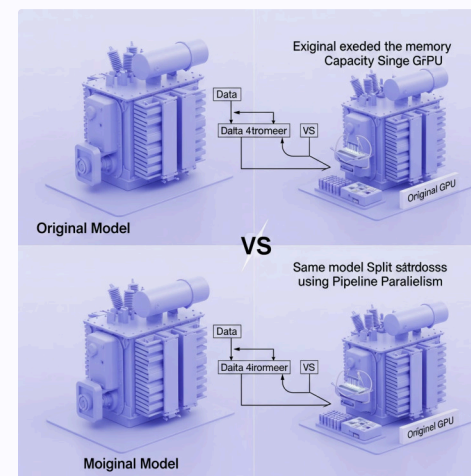
Data parallelism alone creates **redundant parameters** on each GPU, limiting maximum model size

Layer-Based Division

Pipeline parallelism splits **layers into stages** across multiple GPUs, enabling sequential processing

Massive Scale

Enables training models with **hundreds of billions of parameters** through efficient distribution



Core Idea: Divide and Pipeline

Layer Distribution

Divide model layers across multiple GPUs, with each GPU responsible for a sequential subset of the network

Stage Processing

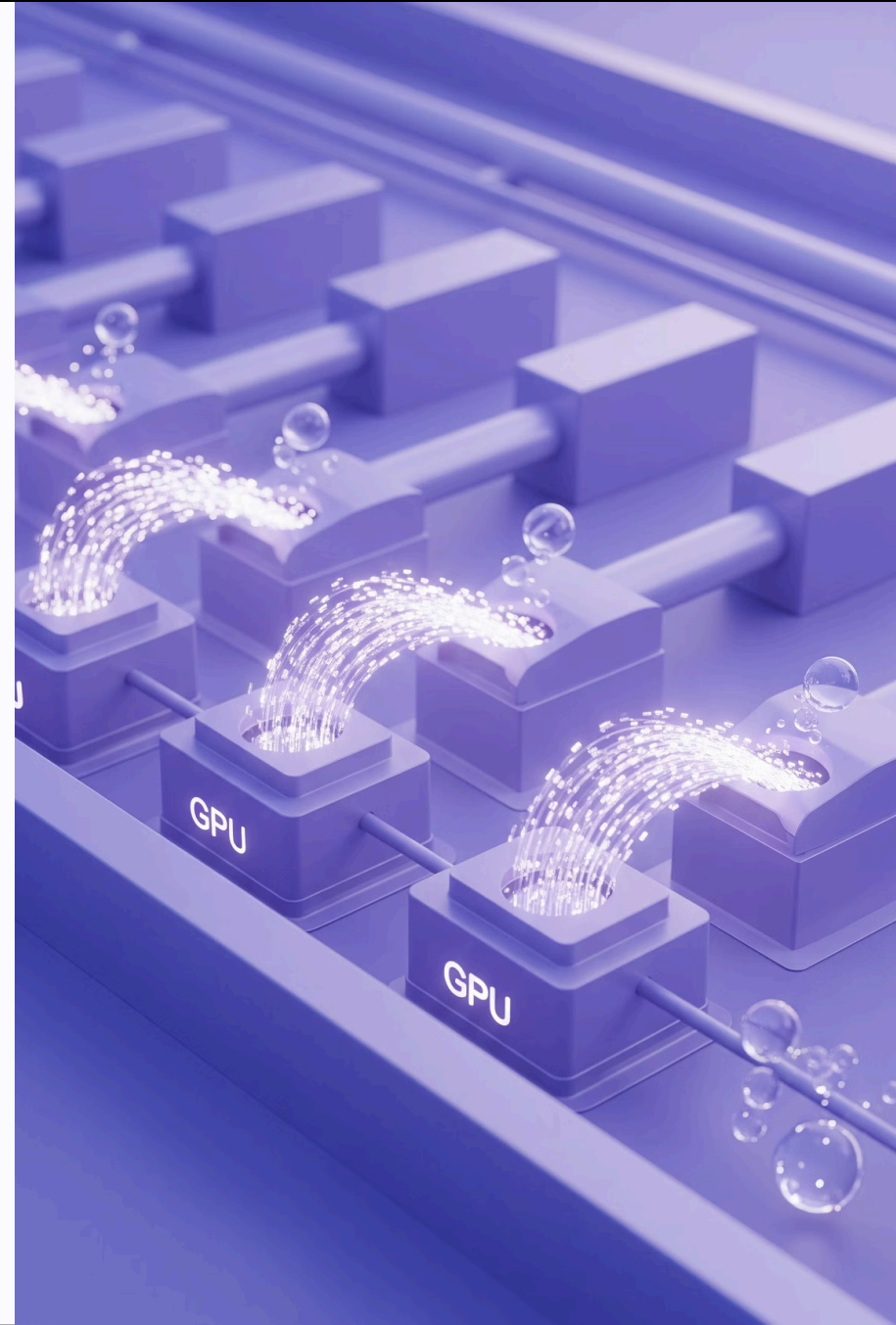
Each GPU stage processes its portion of the forward and backward pass, then transfers activations to the next stage

Micro-Batch Execution

Process small chunks of the global batch to keep all GPUs busy, maximizing hardware utilization

Pipeline Efficiency

Accept the trade-off of "bubble" periods at start and end of pipeline where some GPUs are idle



Example: 4-Stage Pipeline

1

GPU 0: Input Layers

Handles embedding layer and encoder layers 1-6

2

GPU 1: Middle Layers (Lower)

Processes encoder layers 7-12

3

GPU 2: Middle Layers (Upper)

Manages encoder layers 13-18

4

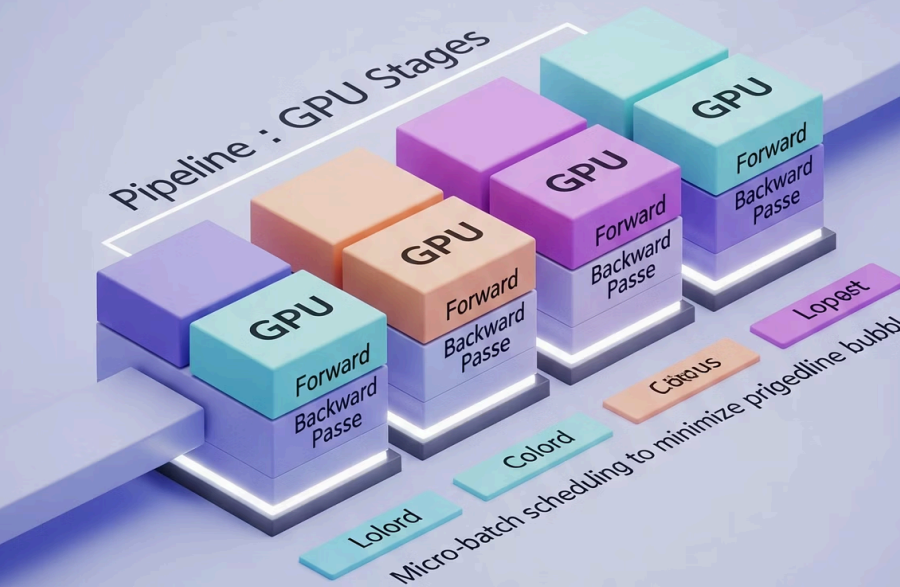
GPU 3: Output Layers

Completes processing with encoder layers 19-24 and output head

Data flows sequentially through each stage, with activations passing from one GPU to the next.



Micro-Batching & Scheduling



Key Concepts

- Split global batch (e.g., 1024) into smaller **micro-batches** (e.g., 8 batches of 128)
- Stagger micro-batch execution across pipeline stages
- Allows multiple micro-batches to be processed simultaneously
- Common scheduling algorithms optimize pipeline efficiency

Scheduling Strategies

GPipe (Fill-Drain)

Complete all forwards, then all backwards. Simple but less efficient.

1F1B Schedule

Alternate 1 forward, 1 backward. Better GPU utilization with steady memory usage.

Infrastructure Requirements



Fast Interconnect

NVLink or InfiniBand with high bandwidth (300+ GB/s) to minimize communication bottlenecks between pipeline stages



Balanced Stages

Even computational load across GPUs to prevent any single stage from becoming a throughput bottleneck



Software Frameworks

DeepSpeed and **Megatron-LM** provide optimized implementations for pipeline parallelism



3D Parallelism

Combine with Data Parallelism (DP) and Tensor Parallelism (TP) for maximum scaling efficiency

Benefits of Pipeline Parallelism



Memory Scaling

Train models far beyond single-GPU memory capacity - memory requirement scales almost linearly with number of pipeline stages



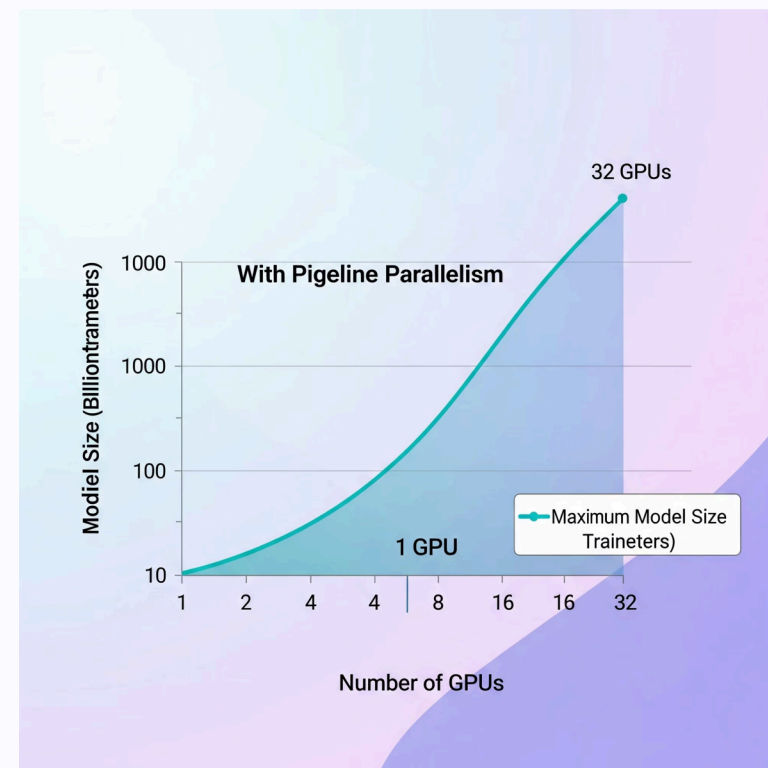
Architecture Flexibility

Compatible with both encoder-only architectures (BERT) and decoder-only models (GPT)



Massive Model Training

Enables efficient training of 100B+ parameter models that would be impossible without specialized parallelism



Challenges

Load Balancing

Uneven computational complexity between stages creates GPU idle time, reducing overall efficiency

Example: Earlier transformer layers may process faster than later layers

Pipeline Bubbles

Unavoidable periods where GPUs must wait for activations or gradients, reducing theoretical throughput

More pronounced with fewer micro-batches or more pipeline stages

Debugging Complexity

Failures across distributed stages are harder to diagnose and reproduce

Training instability may emerge from inter-stage communication patterns

Checkpoint Management

Saving and restoring model state across multiple GPUs adds implementation complexity

Requires coordinated checkpoint protocols

Advanced Optimizations

1 Activation Checkpointing

Discard and recompute intermediate activations during backward pass to reduce memory footprint by 50-75%

2 Communication Overlap

Overlap communication between stages with computation to hide latency

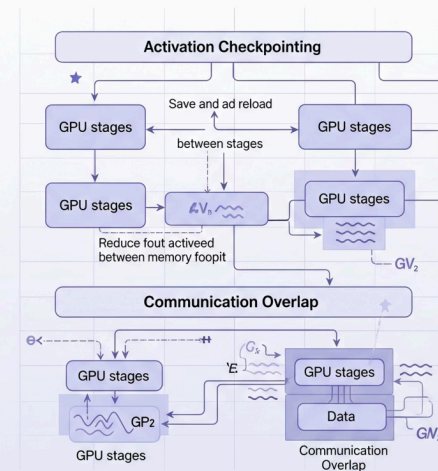
3 Optimized Scheduling

Use 1F1B or interleaved micro-batch schedules to maximize GPU utilization

4 Hybrid Techniques

Combine with [ZeRO/FSDP](#) to reduce parameter redundancy and optimize memory usage

Pipeline Parallelism Optimizations



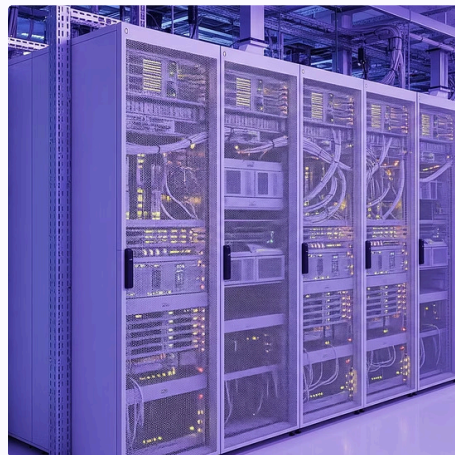
Real-World Use Cases



GPT-3 & MT-NLG

175B parameter models trained using pipeline parallelism across hundreds of GPUs

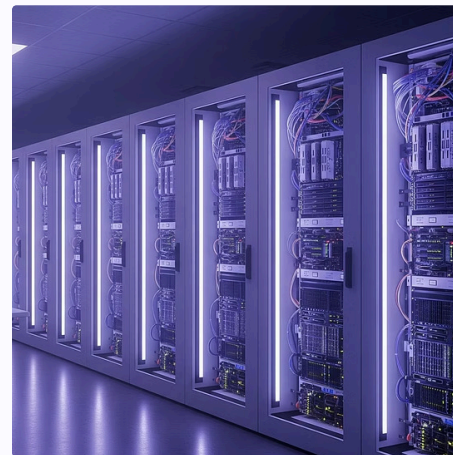
OpenAI and NVIDIA/Microsoft scaled efficiently using DeepSpeed



BLOOM (176B)

Open-science LLM trained using Megatron-DeepSpeed with 3D parallelism

Required pipeline parallelism to fit model across available A100 GPUs



Academic Scaling

Research pushing boundaries toward trillion-parameter models

Pipeline parallelism critical for efficient scaling on limited hardware



Enterprise Models

Custom foundation models trained on industry-specific data

Pipeline parallelism enables scaling within fixed infrastructure budgets



Best Practices

Profile & Balance

- Measure per-layer FLOPs and memory usage to create balanced pipeline stages
- Consider layer characteristics - attention layers vs. feed-forward networks have different computational profiles
- Group layers that have data dependencies to minimize communication

Start Simple, Then Scale

- Begin with fewer pipeline stages (2-4) before attempting larger configurations
- Validate correctness at each scaling step

Batch Size Optimization

- Use synthetic data to tune global batch and micro-batch sizes
- Larger micro-batches increase efficiency but require more memory
- Find sweet spot between pipeline efficiency and memory usage

Performance Monitoring

- Track **tokens/sec/GPU** as primary efficiency metric
- Monitor inter-GPU communication bandwidth utilization
- Check for GPU imbalances in utilization

Key Takeaways

Pipeline Parallelism: Layer-by-Layer Distribution



Model Splitting

Pipeline parallelism divides transformer layers across multiple GPUs, enabling training of models that exceed single-device memory



Micro-Batching

Micro-batching is essential to minimize idle time and maximize hardware utilization across pipeline stages



3D Parallelism

Best results come from combining pipeline with **data and tensor parallelism** for comprehensive scaling



Trillion-Parameter Scale

Pipeline parallelism is a foundational technique for training the largest transformer models, enabling trillion-parameter scale

Pipeline parallelism is now a standard component in large-scale transformer training infrastructure, allowing researchers and engineers to train ever-larger models efficiently.